

Done by: Mostafa Tamer Abuelfadl

Presented to: Eng. Mohammed El Adaway

4-bit ALU Verification Plan

4-bit Arithmetic Logic Unit

- Reset signal RST is asynchronous, active low
- Two WIDTH-bit operands, A and B
- 2-bit opcode selects the operation: ADD, SUB, AND, OR
- Result is a (4-bit) WIDTH-bit registered output, updated on positive edge of CLK
- Carry_Flag is active high for ADD/SUB when operation produces carry-out/borrow beyond MSB; registered
- Arithm_Flag is active high when opcode is ADD or SUB; registered
- Logic_Flag is active high when opcode is AND or OR; registered
- Zero_Flag is active high when $\text{Result} == 0$; combinational derived from registered Result
- On reset ($\text{RST} = 0$), Result, Carry_Flag, Arithm_Flag, and Logic_Flag are all forced to 0

Signals

1. RST → Asynchronous, active low; all registered output signals (Result, Carry_Flag, Arithm_Flag, Logic_Flag) must be forced to zero immediately, independent of CLK

2. A, B → WIDTH-bit operands; driven with valid constrained-random or corner values

3. OP → 2-bit operation select, sampled at posedge CLK:

ADD → $\text{Result} \leq A + B$; Carry_Flag $\leq$ carry-out of addition

SUB → $\text{Result} \leq A - B$; Carry_Flag $\leq$ borrow-out of subtraction

AND → $\text{Result} \leq A \& B$; Carry_Flag ≤ 0

OR → $\text{Result} \leq A | B$; Carry_Flag ≤ 0

4. Result → Checker (golden reference model):

If reset asserted → check $\text{Result} == 0$

Else, at posedge CLK, check Result equals the result of operation selected by opcode on sampled A/B

5. Carry_Flag → Checker:

If reset asserted → check Carry_Flag == 0

If opcode is ADD or SUB → check Carry_Flag equals reference carry/borrow bit

If opcode is AND or OR → check Carry_Flag == 0

6. Arithm_Flag → Checker:

If reset asserted → check Arithm_Flag == 0

If opcode is ADD or SUB → check Arithm_Flag == 1 on following posedge CLK

Otherwise → check Arithm_Flag == 0

7. Logic_Flag → Checker:

If reset asserted → check Logic_Flag == 0

If opcode is AND or OR → check Logic_Flag == 1 on following posedge CLK

Otherwise → check Logic_Flag == 0

8. Zero_Flag → Checker:

If Result == 0 → check Zero_Flag == 1

Else → check Zero_Flag == 0

Sampled combinationaly, tracking registered Result

Functional Coverage

Cover all output signals and controlling inputs:

- A → Cover corner values: A == 0 (ZERO), A == max value (MAX), and default (all other values)
- B → Cover corner values: B == 0 (ZERO), B == max value (MAX), and default (all other values)
- A (logical ops only) → Cover walking-ones patterns on A {1,2,4,8}, sampled only when opcode is AND or OR
- OP → Cover ADD, SUB (arithmetic bin) and AND, OR (logic bin)
- OP → Cover transition sequence ADD => SUB => AND => OR
- Cross OP × A → Every opcode exercised against every A corner bin

- Cross $OP \times B \rightarrow$ Every opcode exercised against every B corner bin
- Carry_Flag $\rightarrow$ 1 iff an ADD/SUB produces a carry-out/borrow
- Arithm_Flag $\rightarrow$ 1 iff opcode is ADD or SUB
- Logic_Flag $\rightarrow$ 1 iff opcode is AND or OR
- Zero_Flag $\rightarrow$ 1 iff Result == 0, covered across all four opcodes
- Coverage is sampled only when RST == 1 (samples during reset are excluded)

Sequences & Constrained Randomization

1. Reset sequence $\rightarrow$ RST driven low with weight 3/100 per transaction, deasserted with weight 97/100 (reset is rare, matching realistic operating conditions)
2. Arithmetic operand sequence (OP = ADD/SUB) $\rightarrow$ A and B independently weighted: MAX and ZERO each at weight 90, remaining mid-range values collectively at weight 10 — biasing toward carry/overflow and zero-result corner cases
3. Logical operand sequence (OP = AND/OR) $\rightarrow$ A constrained to a walking-ones value {1,2,4,8} with weight 80, or non-walking-ones with weight 20; B constrained $\neq$ A
4. Directed reset transaction $\rightarrow$ First transaction of every run forces RST = 1, A = 0, B = 0, OP = ADD to establish a known post-reset baseline

Test Scenarios

Scenario 1: Full Constrained-Random Regression

Description: 200+ back-to-back transactions, opcode freely randomized across ADD, SUB, AND, OR each cycle

Goal: Exercise every opcode against corner and default operand values, drive functional coverage model to closure, stress Carry_Flag/Zero_Flag checkers under sustained randomization

Scenario 2: Carry/Borrow Stress

Description: ADD/SUB transactions with A, B pinned at MAX/ZERO (weight-90 bins)

Goal: Force repeated carry-out and borrow-out events on Carry_Flag and confirm Zero_Flag asserts correctly when ADD/SUB result lands on zero

Scenario 3: Logical Walking-Ones Sweep

Description: AND/OR transactions with A driven through walking-ones set {1,2,4,8} and B != A

Goal: Cover every single-bit A pattern crossed with AND/OR, confirm Logic_Flag == 1 and Carry_Flag == 0 throughout

Scenario 4: Opcode Transition Coverage

Description: Sequence ADD => SUB => AND => OR repeated with random operands

Goal: Close opcode transition bin and confirm Arithm_Flag/Logic_Flag switch correctly one cycle after each opcode change

Scenario 5: Asynchronous Reset Scenario

Description: RST toggled low at random points during active randomization (weight 3/100 per transaction)

Goal: Confirm Result, Carry_Flag, Arithm_Flag, Logic_Flag clear immediately on reset assertion regardless of CLK, and operation resumes correctly once RST deasserts